

Experimenting Neural Networks on Scratch and PyTorch Implementations

ARJUN GUPTA

May 30, 2026

1 Introduction

This report presents a comparative study of a multilayer neural network trained on the MNIST handwritten digit dataset using different implementations and hardware configurations. The objective is to analyze the impact of framework-level optimizations and hardware acceleration on convergence speed, memory consumption, and classification performance. Across the experiments carried out, different computation sources have been used.

2 Experimental Results

The experiments were conducted using the following implementations:

1. PyTorch implementation executed on an NVIDIA T4 GPU.
2. PyTorch implementation executed on CPU.
3. Scratch implementation developed using NumPy and executed on a local machine.
4. Scratch implementation executed on Google Colab CPU.

2.1 Performance Comparison

Table 1: Performance Comparison Across Implementations

Implementation	Training Time (s)	Peak Memory (MB)	Test Accuracy (%)
PyTorch (T4 GPU)	460.01	1872.71	93.35
PyTorch (CPU)	597.95	1218.66	93.54
Scratch (Local)	846.92	1028.67	94.61
Scratch (Colab CPU)	2288.18	1197.30	94.60

3 Analysis and Discussion

3.1 Convergence Speed

The results demonstrate significant differences in convergence speed across the evaluated implementations. The PyTorch implementation executed on the NVIDIA T4 GPU achieved the fastest training time of 460.01 seconds. The PyTorch CPU implementation completed training in 597.95 seconds, while the locally executed scratch implementation required 846.92 seconds. The slowest execution was observed for the scratch implementation on Google Colab CPU, which required 2288.18 seconds.

Furthermore, PyTorch utilizes highly optimized libraries such as CUDA and cuDNN, which provide efficient implementations of tensor operations and automatic differentiation. These optimizations significantly reduce computational overhead and improve hardware utilization.

It is essential to note here that the cost functions did not actually converge as they were only optimized across 3000 iterations, as in earlier experimentation with different parameters the training time taken was coming out to be 60 minutes or more due to which I had to suffice for smaller number of iterations.

Despite that, decent test accuracies were still obtained, which could be further improved by training the models for longer periods of time.

3.2 Memory Usage Analysis

Peak memory consumption varied considerably across the implementations.

The PyTorch GPU implementation exhibited the highest memory consumption at 1872.71 MB. This is expected because GPU execution requires additional memory for tensors, computational graphs, gradients, optimizer states, and intermediate activations.

The PyTorch CPU implementation consumed 1218.66 MB, while the scratch implementations consumed 1028.67 MB and 1197.30 MB for local and Colab execution, respectively.

The lower memory footprint of the scratch implementation can be attributed to its simpler execution model. Since gradients and activations are managed manually, there is no need for automatic differentiation structures or tracking of the state of the optimizer. Although this reduces memory usage, it comes at the cost of reduced computational efficiency and longer training times.

3.3 Classification Performance

The scratch implementation achieved the highest classification performance, obtaining a test accuracy of 94.61% on the local machine and 94.60% on Google Colab.

The PyTorch implementations achieved test accuracies of 93.35% and 93.54% on GPU and CPU respectively.

The relatively small difference in accuracy indicates that all implementations successfully learned useful representations of the MNIST dataset. The high accuracies obtained across all experiments demonstrate the effectiveness of the chosen neural network architecture for handwritten digit classification.

3.4 Confusion Matrix Analysis

The confusion matrices obtained from all implementations exhibit strong diagonal dominance, indicating that the majority of test samples were classified correctly.

The most frequent misclassifications occurred between visually similar handwritten digits such as:

- Digits 3 and 5
- Digits 4 and 9
- Digits 5 and 8
- Digits 7 and 9

These errors are expected because handwritten digits often contain ambiguous shapes and stylistic variations that make certain classes difficult to distinguish.

Despite these challenges, all implementations achieved strong classification performance and demonstrated good generalization on the MNIST test set.

3.5 Effect of Hardware Differences

A notable observation is the substantial difference in execution time between the locally executed scratch implementation and the Google Colab scratch implementation, despite both executing identical algorithms.

The local implementation was executed on an Intel Core Ultra 9 275HX processor, whereas the Google Colab implementation utilized an Intel Xeon CPU operating at 2.20 GHz.

The Intel Core Ultra 9 275HX provides significantly higher computational performance due to:

- Higher instruction throughput
- More advanced processor architecture
- Better single-thread performance
- Improved cache hierarchy and memory subsystem

As a result, the local execution completed training approximately 2.7 times faster than the Colab CPU implementation.

Interestingly, despite the large difference in execution time, both implementations achieved nearly identical test accuracies. This observation demonstrates that hardware primarily affects training speed rather than the final predictive capability of the trained model.

3.6 Framework Optimizations and Numerical Efficiency

PyTorch provides several advantages that contribute to improved computational efficiency:

- Automatic differentiation through Autograd
- Optimized tensor computation kernels
- Efficient memory management
- GPU acceleration support
- Highly optimized backend libraries

These features allow PyTorch to achieve significantly faster convergence compared to manually implemented neural networks.

The scratch implementation offers complete transparency into the learning process and serves as an effective educational tool for understanding the mechanics of neural network training. However, this transparency comes at the cost of increased implementation complexity and reduced computational efficiency.

3.7 Overall Discussion

The experimental results clearly demonstrate the importance of both software framework optimizations and hardware acceleration in deep learning.

Among all evaluated configurations, the PyTorch implementation executed on the NVIDIA T4 GPU achieved the fastest convergence due to efficient utilization of massively parallel GPU resources and optimized backend libraries.

The scratch implementation achieved the highest classification accuracy, albeit with substantially longer training times. This highlights the trade-off between implementation transparency and computational efficiency.

Overall, PyTorch offers the most practical solution for large-scale neural network training due to its optimized execution model, automatic differentiation capabilities, and support for hardware acceleration. The scratch implementation remains valuable for educational purposes and for developing a deeper understanding of the underlying principles of neural network learning.

4 Conclusion

This study compared neural network training across multiple frameworks and hardware configurations using the MNIST dataset.

The results showed that PyTorch significantly improves training efficiency through framework-level optimizations and hardware acceleration. GPU execution provided the fastest convergence, while CPU execution offered a balance between memory consumption and training speed.

The scratch implementation achieved slightly higher classification accuracy but required substantially longer training times. Hardware differences between the Intel Core Ultra 9 275HX processor and the Intel Xeon CPU used in Google Colab further demonstrated the impact of processor architecture on execution speed.

Overall, the experiments highlight the importance of both software optimizations and hardware capabilities in determining the performance of deep learning systems. Please note that LLMs were used for formatting and refining this document.